Your game.
Your price.

A private Steam watchlist with target-price alerts.

INDIVIDUAL STEAM APP / TYPE: GAME / EXACT US · USD OFFER

Steam’s game-type check cannot distinguish every special edition.

Five people, one product

Alex	Lead, integration & Flutter
Martin	React web app & interface
Ibrahim	MongoDB, infrastructure & tests
James	Authentication & shared API
Nick	Game matching & notification adapters

Two clients. One shared API.

CLIENTS

React + Tailwind
TypeScript
Flutter / Android

→

SHARED API

Express · Node.js
TypeScript

→

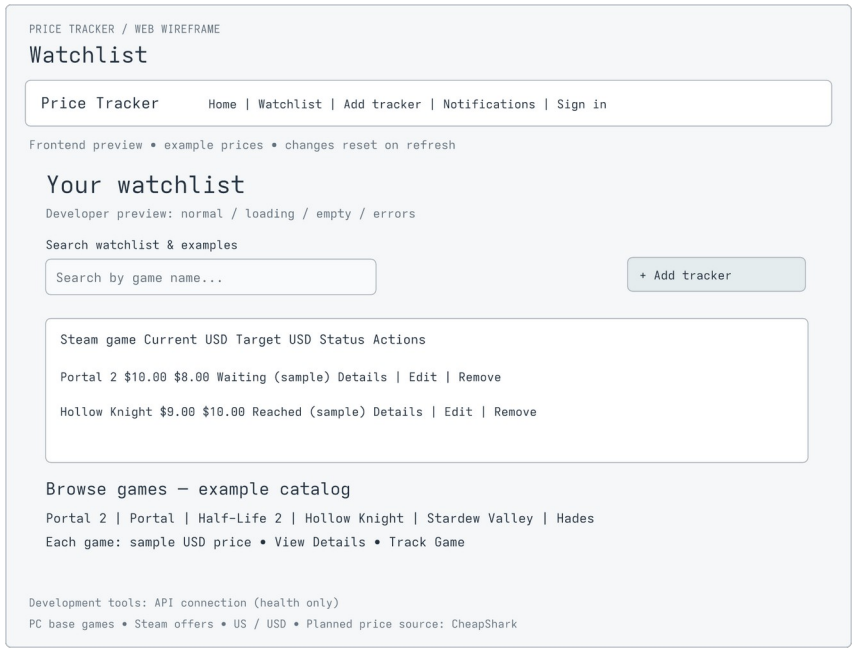
DATABASE

MongoDB
Mongoose

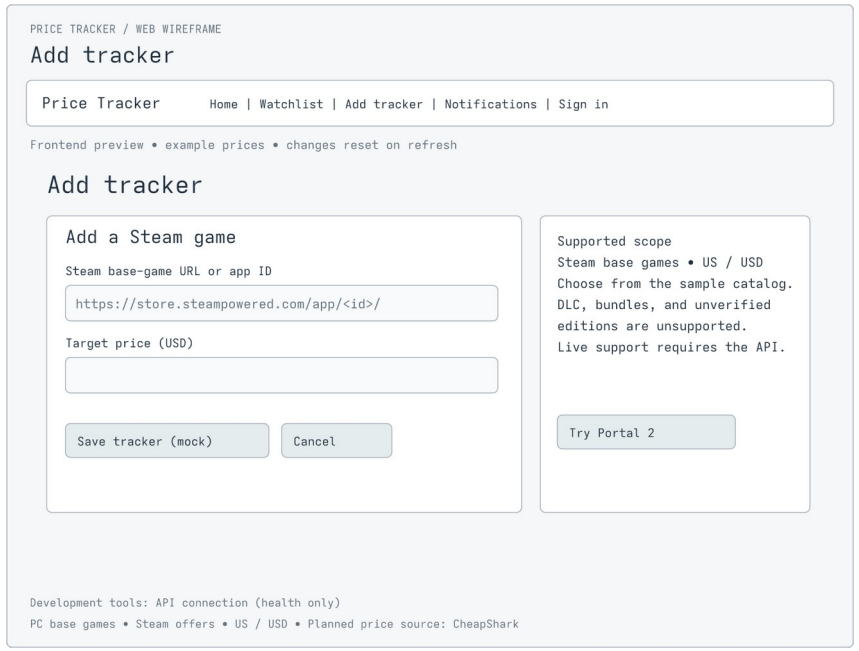
CheapShark → Steam offers | Resend + Firebase → alerts

Tools: GitHub + Actions • OpenAPI / SwaggerHub publication pending

Design the flow before the details



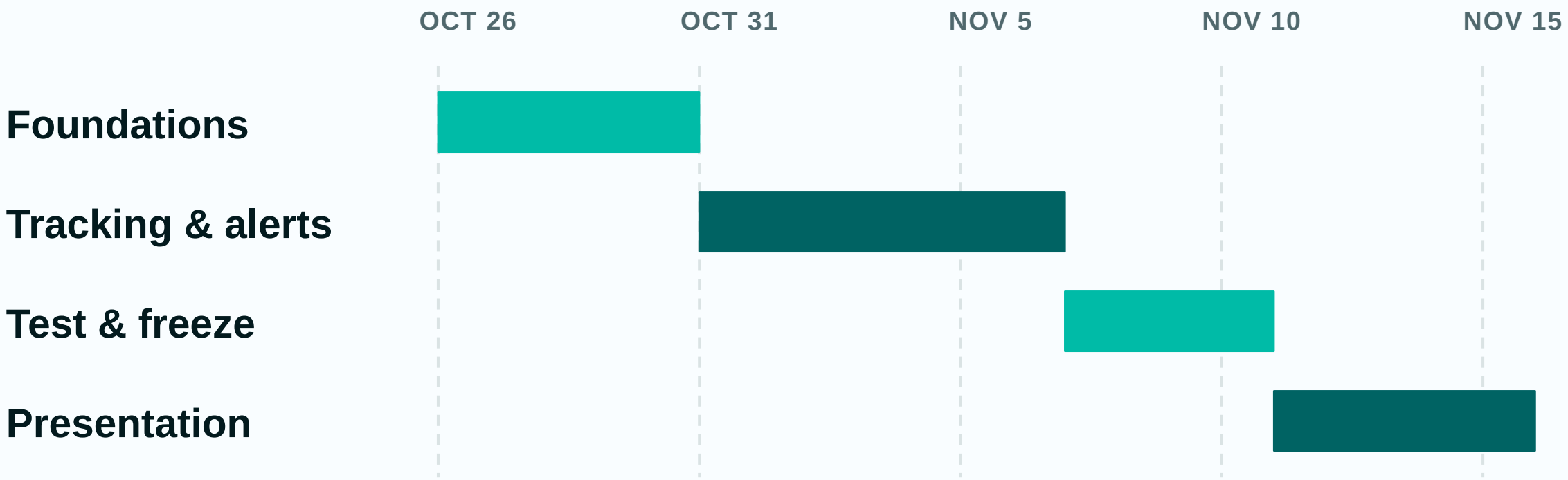
01 / WATCHLIST



02 / ADD GAME

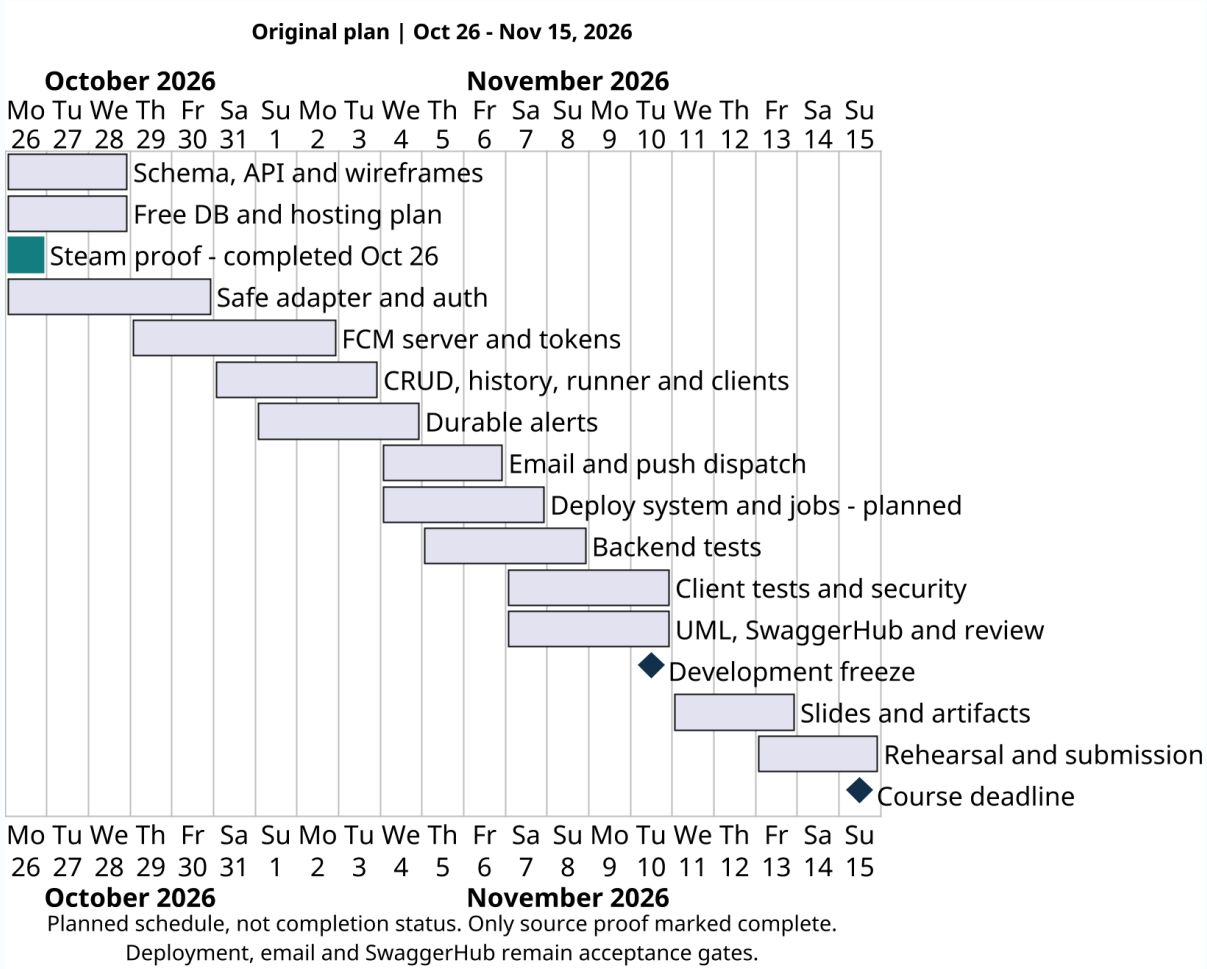
Early web planning artifacts; sketch prices are illustrative.

From foundation to presentation

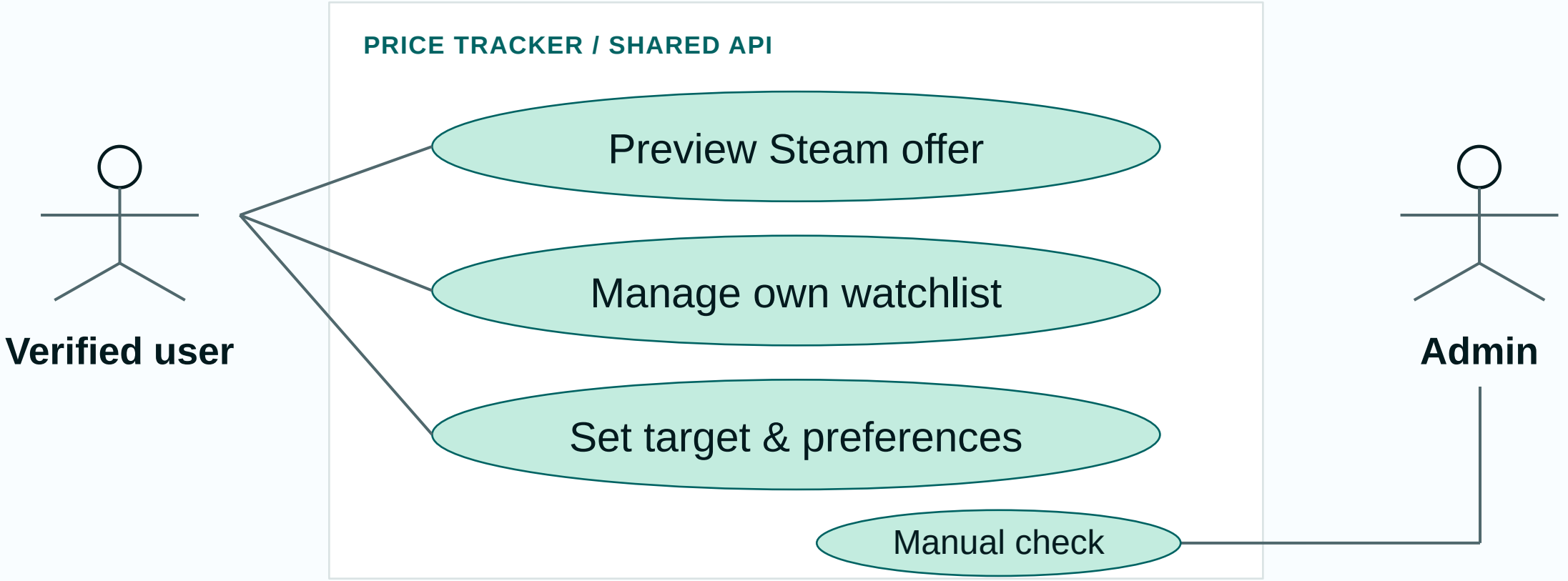


Original planned dates, Oct 26–Nov 15; detailed tasks follow.

How the planned work overlapped

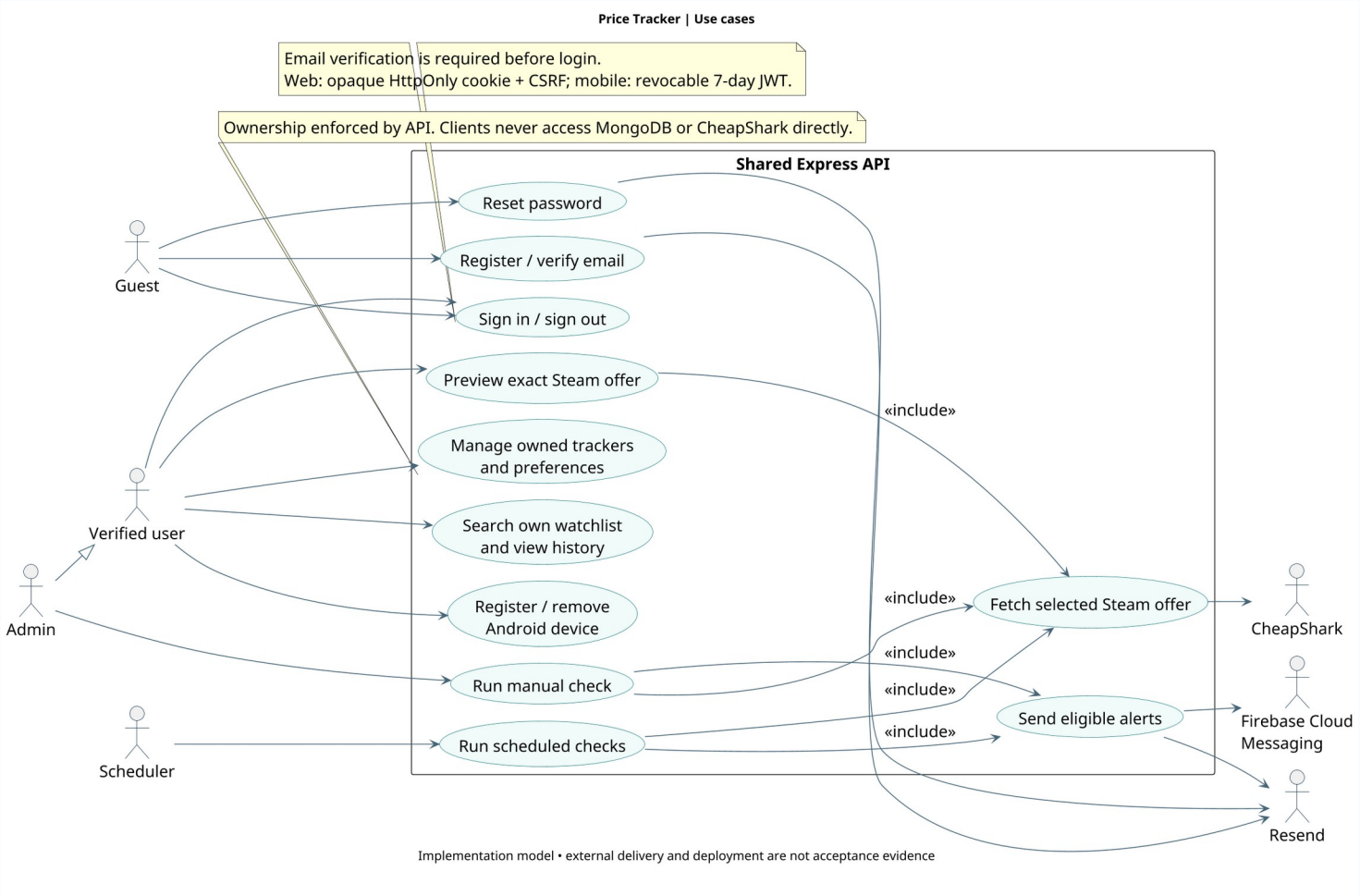


A private watchlist for each player

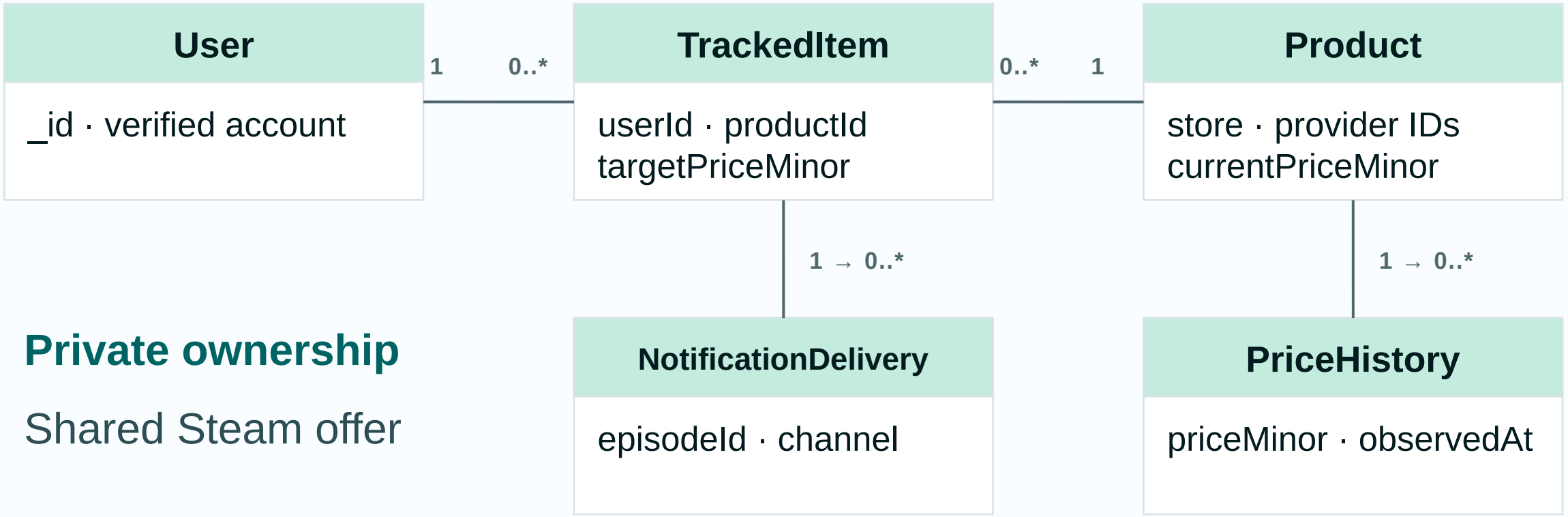


Core use cases; the next slide includes authentication and external actors.

Actors and external services



Shared prices. Private targets.

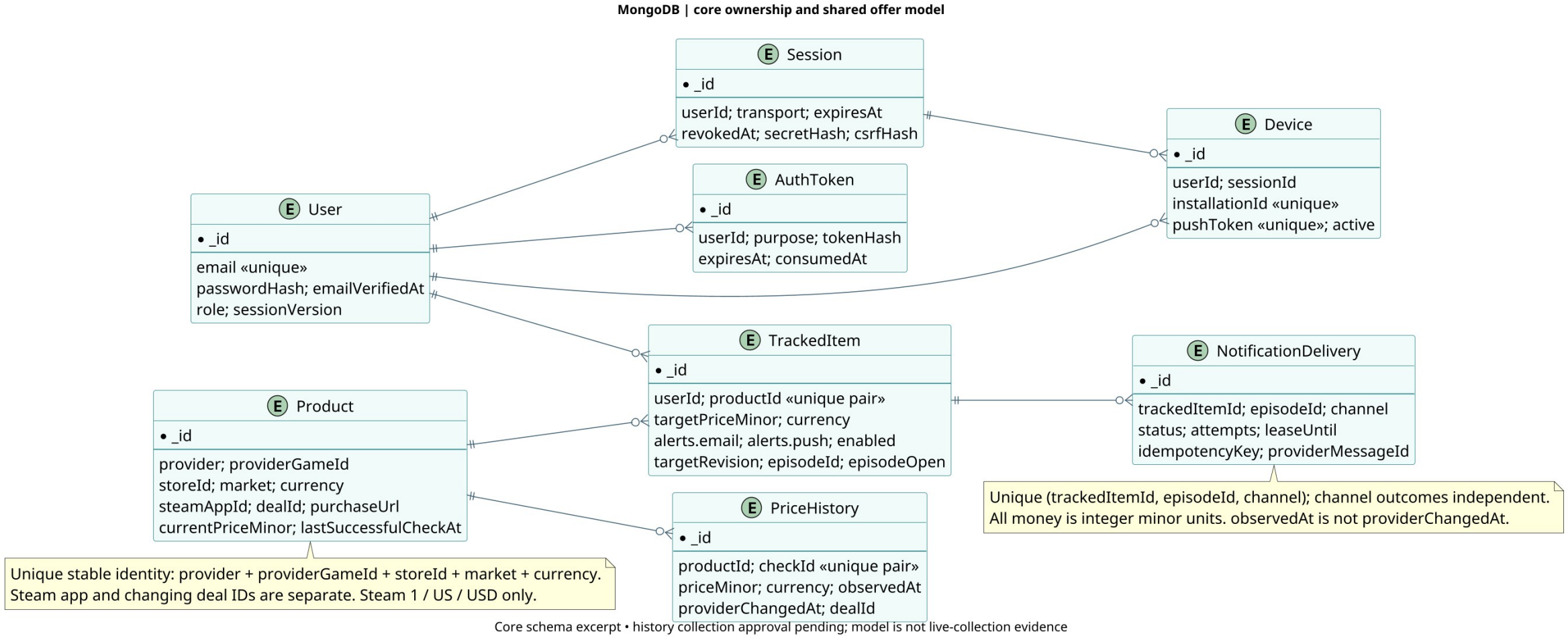


Private ownership

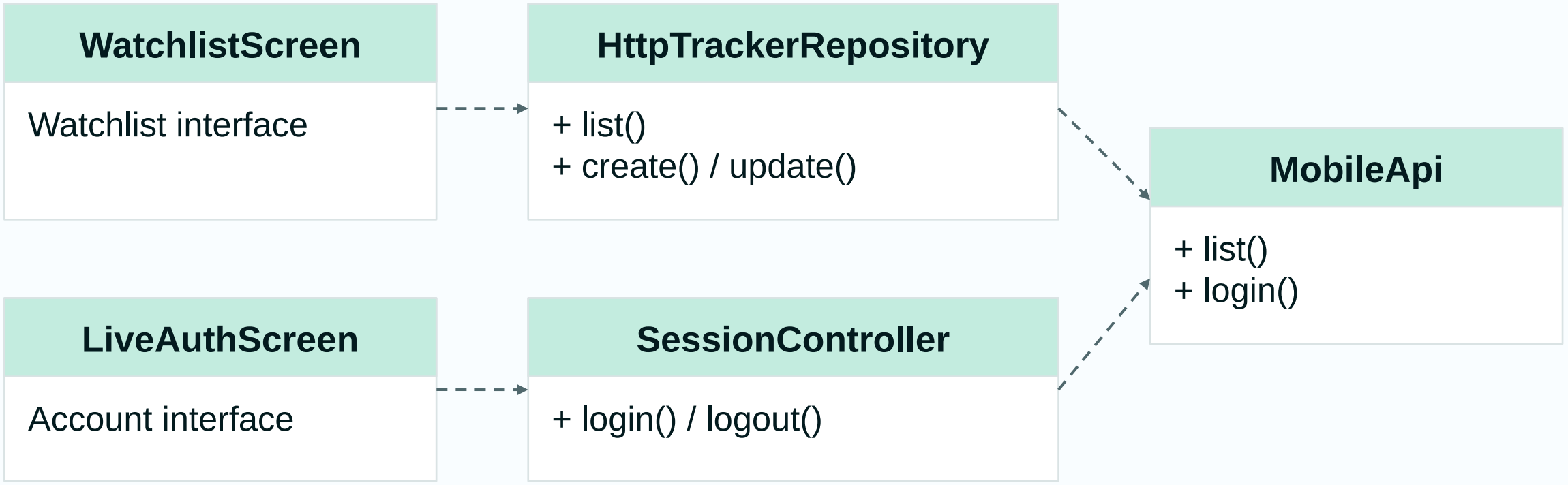
Shared Steam offer

Core ERD excerpt • USD amounts use integer cents.

Accounts, sessions, and delivery records

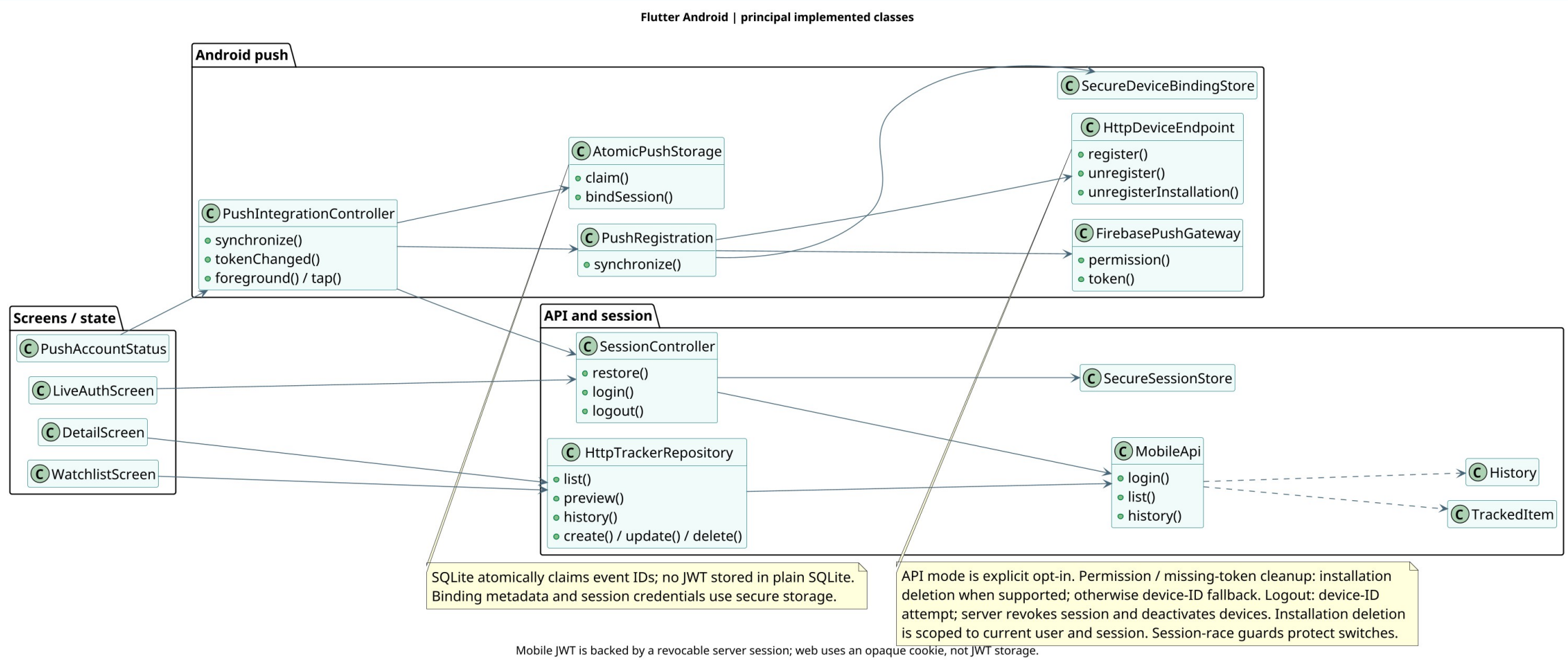


Mobile screens stay separate from data

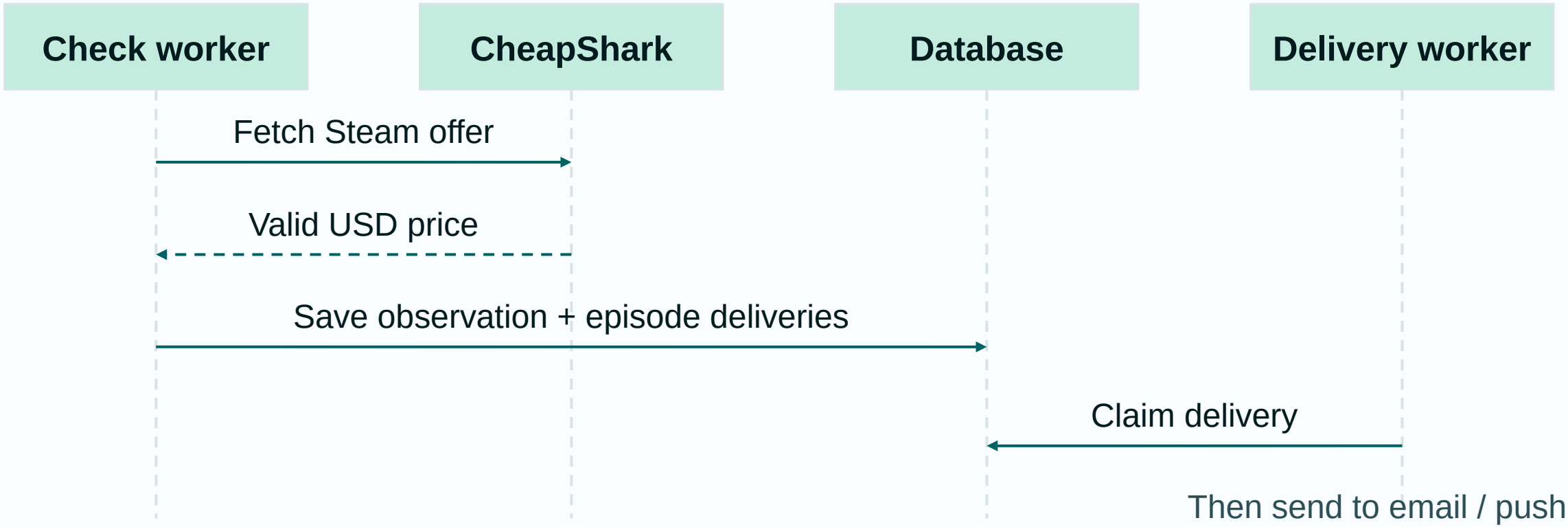


Core dependencies; the next slide includes secure storage and push classes.

Mobile sessions and device lifecycle

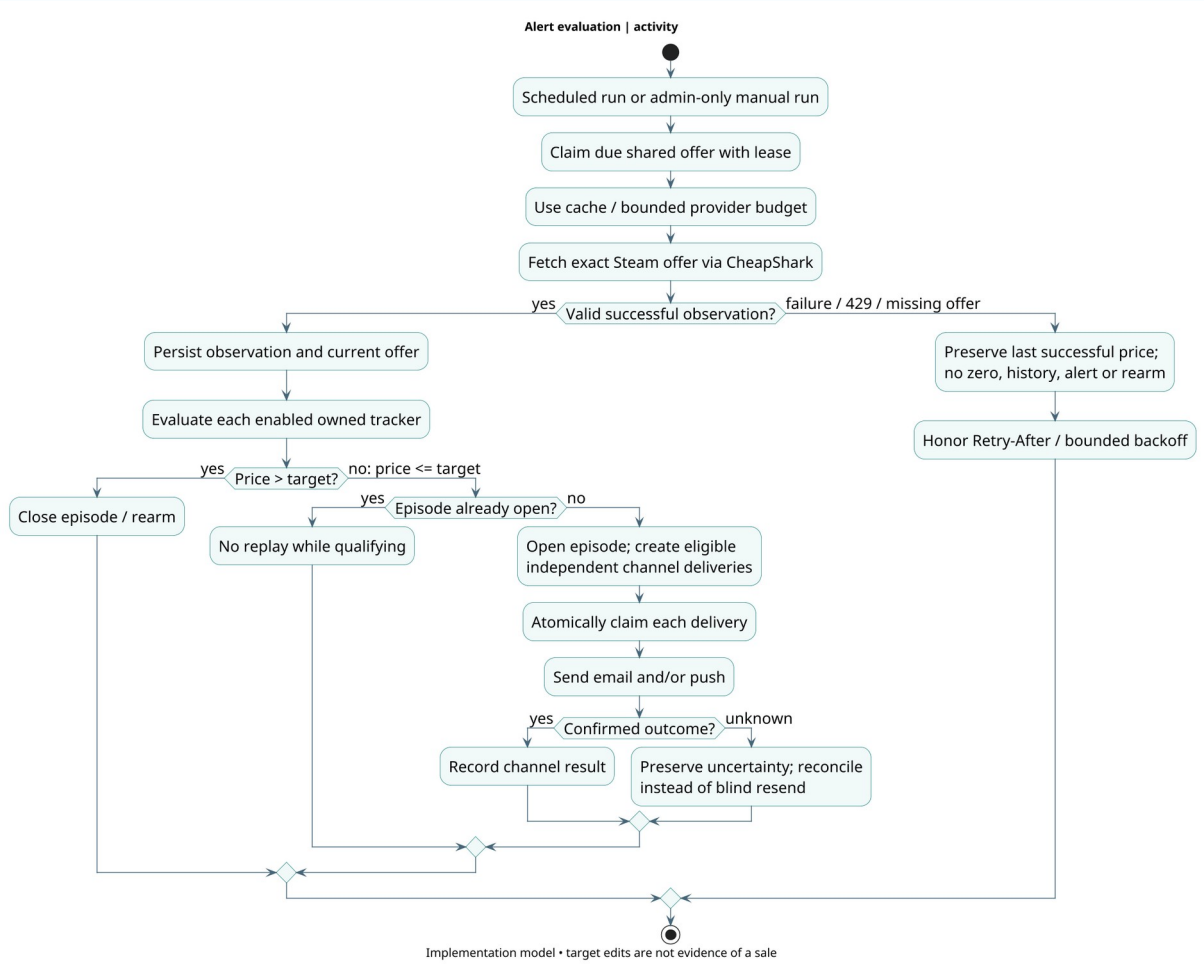


Check. Record. Deliver independently.

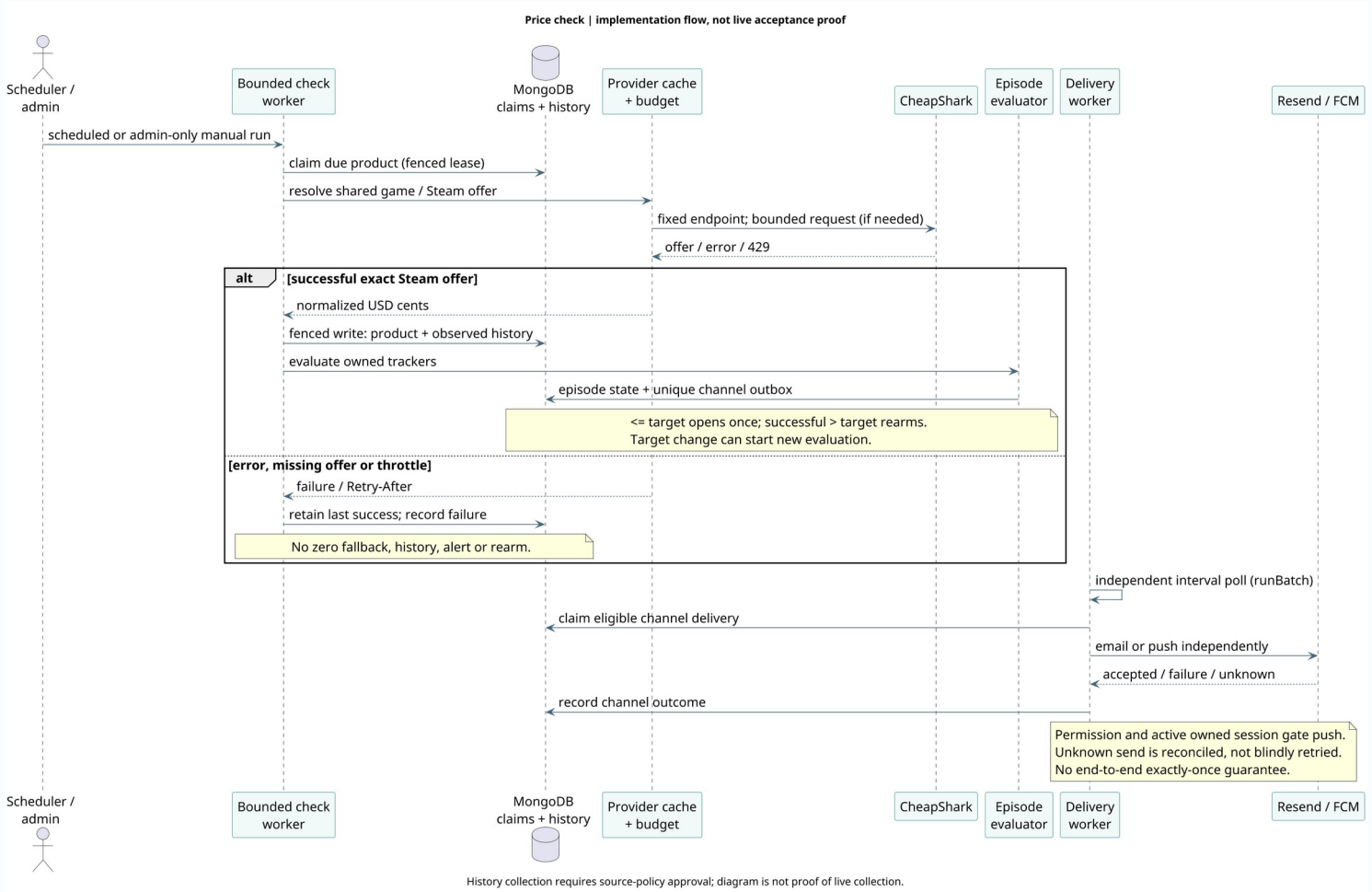


Sequence excerpt • Delivery polls independently; failures preserve the last price.

When an alert qualifies



From provider response to delivery outcome



Test results, with clear limits

NODE / RECORDED NOV 11

735 passing tests
Zero skips

FLUTTER / RECORDED NOV 11

239 passing tests
Analysis + format passed

Unit + API integration: auth, ownership, tracking, delivery

Manual integration: local web / Android checks

Full hosted mailbox and qualifying-alert acceptance remain open.

LIVE DEMONSTRATION

API demo: search your watchlist

01 / EXECUTE

```
GET /api/tracked-items?q=por&limit=20
```

02 / SHOW

Partial title match

03 / EXPLAIN

Only this user’s records

Rehearsal prerequisite: published SwaggerHub + verified live endpoint.

Web demo: save a game, set a target

01 / FIND

Open the watchlist.
Search a saved title.

02 / ADD

Preview a Steam URL.
Set a target.

03 / EDIT

Save preferences.
Change the target.

Use the hosted API-mode app and a privately signed-in verified account.

Android demo: the same saved game

01 / RELOAD LIST

Open the same account on Android.

02 / COMPARE

Show the tracker just edited on web.

03 / EXPLAIN

Review details and alert preferences.

Target edit changes the threshold; a successful check is needed. No sale is implied.

What worked. What challenged us.

WENT WELL

One API for both clients
Repeatable fixture tests
Cross-client privacy checks

DID NOT GO WELL

Scope changed mid-project
Provider limits and matching
Android lifecycle races

Next lesson: schedule live integration evidence earlier.

Questions?

Design. Testing. Demo.

A Steam watchlist shared across web and Android.

github.com/aluciencozy/price-tracker